Supplementary Implementation Guide

Unsupervised Neural Network for Multi-Genre Music Generation

This guide explains each practical step from dataset loading through model training to metric computation. It is a conceptual companion to your implementation, covering what to do at each stage, why each decision matters, and where to find supporting resources. No code is included; refer to library documentation and the referenced repositories for syntax.

Environment and Libraries

The following libraries are required across all four tasks.

pretty_midi is the standard library for reading and writing MIDI files in Python. It parses a MIDI file into a Python object from which you can access lists of notes, each carrying a pitch number (0–127), a velocity (0–127), a start time, and an end time in seconds. It also provides a method to convert a MIDI file directly into a piano-roll matrix. Documentation: `https://github.com/craffel/pretty-midi`.

miditok converts a MIDI file into a flat sequence of integer tokens following one of several named schemes (REMI, Structured, TSD, and others). This is the standard input format for Transformer models. Documentation: `https://github.com/Natooz/MidiTok`.

PyTorch is the deep learning framework for building, training, and evaluating all four models. It provides tensor operations, automatic differentiation, `Dataset` and `DataLoader` abstractions, and a library of neural network modules. Install from `https://pytorch.org`.

NumPy and pandas handle numerical array operations and the MAESTRO metadata CSV respectively.

Matplotlib produces the loss curve plots required as deliverables.

music21 is an optional library from MIT for music theory analysis: key estimation, chord labeling, and scale analysis. Install from `https://web.mit.edu/music21/`.

> Install PyTorch first using the official selector at `https://pytorch.org/get-started/locally/`, as the correct command depends on your OS and whether a CUDA GPU is available. Install remaining libraries with `pip` afterwards.

Step 1: Loading the MAESTRO Dataset

Dataset Overview

MAESTRO (MIDI and Audio Edited for Synchronous TRacks and Organization) contains approximately 200 hours of classical piano performances recorded at the International Piano-e-Competition. Performers play on Yamaha Disklavier grand pianos, which capture MIDI data with approximately 3 millisecond timing accuracy.

Download the **MIDI-only package** (approximately 57 MB compressed, 85 MB uncompressed). It contains 1,276 MIDI files. The 103 GB full dataset includes audio recordings, which are not needed for any of the four tasks. Download page: `https://magenta.tensorflow.org/datasets/maestro`.

Dataset Structure

After extracting the archive you will find a CSV metadata file named `maestro-v3.0.0.csv` and subdirectories named by competition year (e.g., `2004/`, `2006/`, ..., `2017/`), each containing `.midi` files.

The CSV has one row per recording. The important columns are listed below.

Column	Meaning
`canonical_composer`	Composer name (e.g., Beethoven, Chopin)
`canonical_title`	Piece title
`split`	`train`, `validation`, or `test`
`year`	Competition year
`duration`	Piece duration in seconds
`midi_filename`	Relative path to the `.midi` file

The dataset provides a predefined train/validation/test split across competition years, ensuring no composer appears in both training and test sets. Use this split as given. Do not re-split randomly, as this introduces data leakage between sets. The training set contains 962 recordings, validation 137, and test 177.

Note Representation

Loading a MIDI file with `pretty_midi.PrettyMIDI()` returns an object containing a list of `Instrument` objects. MAESTRO files have a single instrument representing the piano. From that instrument, you access a list of `Note` objects. Each note has four fields: `pitch` (integer 0–127, where middle C is 60), `velocity` (0–127, keystrike force), `start` (onset in seconds), and `end` (offset in seconds). Piano pitches span MIDI numbers 21 (A0) through 108 (C8), covering 88 keys.

Step 2: Exploratory Data Analysis

Exploratory analysis informs every design decision that follows.

Piece duration. Plot a histogram of piece durations across the full dataset. Most MAESTRO recordings are between 3 and 8 minutes long, with some exceeding 20 minutes. This informs your choice of window size for model input.

Note count per piece. Plot the distribution of total notes per recording. Classical piano music is dense; pieces typically contain several thousand notes.

Pitch distribution. Count how frequently each of the 88 piano pitches appears across a sample of files. The distribution is non-uniform: middle-register pitches (roughly C3–C6) appear far more often than extreme low and high pitches. This context is useful when interpreting pitch histogram similarity scores.

Velocity distribution. Plot the distribution of note velocities. Classical piano music spans a wide dynamic range, with velocities typically between 30 and 110. This informs whether to binarize the piano-roll or retain velocity information.

Sparsity check. Convert one or two files to piano-roll format and compute the fraction of cells that are non-zero. You will find approximately 97–98% of cells are zero (silence). This is the primary cause of training instability in Tasks 1 and 2 and must be addressed during preprocessing.

For the final report, include at least a duration histogram and a pitch distribution histogram as evidence of data understanding.

Step 3: Preprocessing

Preprocessing converts raw MIDI files into arrays or token sequences suitable for model training.

Representation Choice

Two representations are used in this project.

Binary piano-roll is a two-dimensional binary matrix with time steps on one axis and the 88 piano pitches on the other. Each cell is 1 if that pitch is active at that time step and 0 otherwise. The time resolution is set by a frames-per-second parameter (`fs`). At `fs`= 16, one second corresponds to 16 columns. A window of 128 time steps covers 8 seconds of music. The piano-roll is the input format for Tasks 1 and 2.

Token sequences represent music as a flat list of discrete integers, where each integer encodes a musical event. The REMI scheme in `miditok` uses: `Bar`, `Position` (offset within bar), `Pitch`, `Velocity`, and `Duration`. A short phrase becomes a sequence such as `[Bar, Position_0, Pitch_60, Velocity_5, Duration_8, ...]`. Token sequences are the input format for Tasks 3 and 4.

Building Piano-Roll Windows (Tasks 1 and 2)

Processing a MIDI file into training windows involves four stages.

Stage 1: Extract the piano-roll. Use `pretty_midi`'s `get_piano_roll()` method at the chosen frame rate. This produces a matrix of shape $(128, T)$. Slice pitch rows 21 through 108 to retain only the piano range, giving $(88, T)$. Transpose to $(T, 88)$ so that time is the first axis, matching PyTorch's LSTM input convention.

Stage 2: Binarize. Set all non-zero values to 1. This discards velocity information and simplifies the learning problem. Velocity can be incorporated in later iterations.

Stage 3: Segment into windows. Divide the $(T, 88)$ matrix into non-overlapping windows of length 128. Each window is one training sample of shape $(128, 88)$.

Stage 4: Filter sparse windows. Discard windows in which fewer than approximately 2% of cells are active. Near-silent windows provide no useful learning signal and artificially inflate sparsity.

Stack the retained windows into a NumPy array of shape $(N, 128, 88)$ and save to disk. Repeat separately for the validation and test splits.

> At `fs`= 16, 128 time steps covers 8 seconds of music, which is consistent with the project specification's recommendation of 16 steps per bar at moderate tempos. If training is slow, reduce the window length. For longer musical context, increase it.

Building Token Sequences (Tasks 3 and 4)

Select the REMI tokenizer in `miditok` with a moderate number of velocity bins (e.g., 32). Optionally train a BPE vocabulary on your MIDI files, which merges frequent event pairs into single tokens and reduces sequence length. Each file is passed to the tokenizer to produce an integer ID list. Use a custom PyTorch collation function to pad variable-length sequences within each batch and produce an attention mask.

For Transformer training, the input is the token sequence up to position $t - 1$ and the target is the same sequence shifted one position forward. The model learns to predict the next event given all preceding events.

Step 4: PyTorch Dataset and DataLoader

A `Dataset` defines how to access a single sample. For piano-roll data, load the saved NumPy array once at initialization and return the appropriate row by index. For token data, return the appropriate token list.

A `DataLoader` wraps a `Dataset` and handles batching, shuffling, and parallel loading. Set `shuffle=True` for training and `shuffle=False` for validation and test. A batch size of 64 is a reasonable starting point; reduce to 32 if GPU memory is insufficient.

Step 5: Task 1, LSTM Autoencoder

Mathematical Objective

The encoder learns a latent embedding:

$$z = f_\phi(X)$$

The decoder reconstructs the sequence:

$$\hat{X} = g_\theta(z)$$

The reconstruction loss defined in the project specification is:

$$\mathcal{L}_{AE} = \sum_{t=1}^{T} \|x_t - \hat{x}_t\|^2$$

Architecture

The **encoder** processes a piano-roll window of shape $(T, 88)$ through stacked LSTM layers. After the final time step, the hidden state of the last layer is projected to a latent vector z of dimension 64 or 128. This bottleneck compresses the musical sequence.

The **decoder** receives z and reconstructs the original sequence. A practical approach is to repeat z across all T time steps, then pass the resulting $(T, \text{hidden_dim})$ tensor through LSTM layers followed by a linear projection to 88 dimensions. The output shape is $(T, 88)$. Do not apply a sigmoid at the output layer; output raw logits and apply the sigmoid inside the loss function for numerical stability.

A two-layer LSTM with hidden dimension 256 and latent dimension 64 is a reasonable starting configuration.

Loss Function and Sparsity

The reconstruction loss in the project specification is the squared error $\mathcal{L}_{AE} = \sum_{t=1}^{T} \|x_t - \hat{x}_t\|^2$. In practice, this formulation fails on highly sparse piano-rolls because the loss is dominated by silent cells and the model converges to predicting near-zero everywhere without learning any note activations. Plain BCE without class weighting has the same problem: the model settles at a constant-output equilibrium, and the gradient signal from rare note activations is insufficient to move parameters away from it.

Focal Loss is a practical replacement that addresses class imbalance directly. It introduces a modulating factor $(1 - p_t)^\gamma$ that suppresses the gradient contribution of easy examples. For a silent cell correctly predicted as near-zero, the factor approaches zero and it contributes negligible gradient. For a note cell incorrectly predicted as near-zero, the factor approaches one and it receives full gradient signal. An additional positive class weight further compensates for the imbalance between active and silent cells.

Music Generation

To generate new samples, bypass the encoder and sample $z \sim \mathcal{N}(0, I)$ directly. Pass z through the decoder and apply a sigmoid to obtain probabilities. Binarize using a threshold below 0.5: because the model is trained on imbalanced data, it tends to underestimate note probabilities, and a threshold of exactly 0.5 suppresses most generated notes. Convert the binary array to a MIDI file using `pretty_midi`: each active cell at time step t for pitch index p corresponds to a note of pitch $p + 21$ with onset and offset determined by the frame duration $1/\texttt{fs}$.

Required deliverables for Task 1: autoencoder implementation code, reconstruction loss curve, and 5 generated MIDI samples.

Interpreting the Loss Curve

A stable training run shows reconstruction loss decreasing over the first several tens of epochs before plateauing. Validation loss should follow training loss closely. If both losses plateau immediately at a constant value from epoch one, revisit the loss function and positive class weight. If validation loss increases while training loss continues to fall, reduce model capacity or add dropout.

Step 6: Task 2, Variational Autoencoder

Mathematical Objective

The VAE extends Task 1 by treating the latent code as a distribution rather than a point estimate. The encoder outputs a mean $\mu(X)$ and a log-variance $\log \sigma^2(X)$, which together define a Gaussian posterior:

$$q_\phi(z|X) = \mathcal{N}(\mu(X),\, \sigma^2(X))$$

Sampling uses the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

where $\sigma = \exp\big(0.5 \cdot \log \sigma^2\big)$. The total VAE loss is:

$$\mathcal{L}_{VAE} = \mathcal{L}_{\text{recon}} + \beta\, D_{\text{KL}}(q_\phi(z|X) \| p(z))$$

where the KL divergence has the closed form:

$$D_{\text{KL}} = -\frac{1}{2} \sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

Architecture

The encoder is modified to output two vectors of the same dimension: μ and $\log \sigma^2$, each produced by a separate linear layer applied to the LSTM final hidden state. The standard deviation σ is then recovered as $\exp(0.5 \cdot \log \sigma^2)$ before the reparameterization step. The decoder architecture is identical to Task 1.

KL Annealing

A common failure mode called posterior collapse occurs when the decoder becomes powerful enough to reconstruct sequences without using z. The encoder then collapses to the prior and the KL term drops to near zero. The fix is **KL annealing**: start training with $\beta = 0$ (pure reconstruction) and linearly increase β to 1 over a warmup period of several tens of epochs. This ensures the encoder learns a useful representation before the KL term begins constraining it.

Multi-Genre Generation

Because MAESTRO contains only classical piano, Task 2's multi-genre requirement can be addressed by treating different composers or time periods as proxy genres (e.g., Baroque vs. Romantic). Alternatively, include a small number of files from the Lakh MIDI Dataset to broaden the genre coverage. Generate 8 samples by sampling $z \sim \mathcal{N}(0, I)$ and decoding.

Required Deliverables for Task 2

- VAE code with KL-divergence loss
- Multi-genre generation outputs (8 MIDI samples)
- **Latent interpolation experiment:** encode two real pieces to obtain μ_1 and μ_2, then generate 8 intermediate outputs at $z_\alpha = (1-\alpha)\mu_1 + \alpha\mu_2$ for $\alpha \in \{0, \frac{1}{7}, \ldots, 1\}$. Export each as a MIDI file.
- **Metric comparison vs. Task 1:** report reconstruction loss, rhythm diversity, and human score side by side for both models.

Step 7: Task 3, Transformer Generator

Mathematical Objective

The Transformer models the autoregressive factorization:

$$p(X) = \prod_{t=1}^{T} p(x_t | x_{<t})$$

Training minimizes the negative log-likelihood:

$$\mathcal{L}_{TR} = -\sum_{t=1}^{T} \log p_\theta(x_t | x_{<t})$$

Perplexity on held-out data is the primary evaluation metric:

$$\text{Perplexity} = \exp\left(\frac{1}{T}\,\mathcal{L}_{TR}\right)$$

A genre conditioning term from the specification is:

$$h_t = \text{Emb}(x_t) + \text{Emb}(\text{genre})$$

This adds a learnable genre embedding to each token embedding, allowing the model to condition generation on a genre label.

Architecture

Use a decoder-only Transformer (GPT-style). Components are: a token embedding layer mapping vocabulary integers to vectors of dimension d_{model} (e.g., 256); a positional embedding layer; a stack of 4–8 Transformer decoder layers, each with masked multi-head self-attention and a feed-forward sublayer; and a final linear projection to vocabulary size producing logits over the next token.

The causal mask in self-attention prevents each position from attending to future positions, which is required for valid autoregressive generation.

Training

Feed the token sequence $[x_1, \ldots, x_{T-1}]$ as input and $[x_2, \ldots, x_T]$ as targets. Compute average cross-entropy across all positions. Monitor perplexity on the validation set each epoch.

Music Generation

Start with a seed token (e.g., `Bar`) and autoregressively sample the next token from the model's predicted distribution at each step until the desired length is reached. Use `miditok`'s built-in conversion to write the token sequence to a MIDI file.

A direct reference implementing this full pipeline on MAESTRO in Keras (2024) is at: `https://keras.io/examples/generative/midi_generation_with_transformer/`

Required deliverables for Task 3: Transformer implementation, perplexity evaluation report, 10 generated long-sequence compositions, and a baseline comparison table.

Step 8: Task 4, Reinforcement Learning for Human Preference Tuning

Mathematical Objective

Task 4 fine-tunes a pretrained generator (Task 3) using human feedback. A music sample is generated by the current policy:

$$X_{\text{gen}} \sim p_\theta(X)$$

A human evaluator assigns a reward score:

$$r = \text{HumanScore}(X_{\text{gen}})$$

The optimization objective is to maximize the expected reward:

$$\max_\theta \; J(\theta) = \mathbb{E}[r(X_{\text{gen}})]$$

The policy gradient update is:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[r \cdot \nabla_\theta \log p_\theta(X)\right]$$

Setup

Begin from the trained Task 3 Transformer. Do not train Task 4 from scratch. The generator is the policy $p_\theta(X)$ and its parameters are updated using the policy gradient rule above.

Human Reward Collection

Generate a set of music samples using the current model. Convert each to audio for listening (see Section 13). Recruit at least 10 participants to rate each sample on a 1–5 scale. Compute the mean rating per sample; this is the reward r for that generation. You may optionally train a reward model (a small neural network) that predicts human scores from musical features, which allows reward computation without running the survey at every training iteration.

Policy Gradient Training

At each iteration: generate a batch of samples $\{X_{\text{gen}}^{(i)}\}$, collect or predict their reward scores $\{r^{(i)}\}$, compute the REINFORCE gradient estimate $\frac{1}{N}\sum_i r^{(i)} \nabla_\theta \log p_\theta(X^{(i)})$, and update θ by gradient ascent with a small learning rate. Normalize rewards by subtracting the batch mean and dividing by the batch standard deviation to reduce variance.

Required deliverables for Task 4: a human listening survey dataset (minimum 10 participants), a reward scoring function or model, 10 RL fine-tuned music outputs, and a before/after comparison of all metrics.

Step 9: Evaluation Metrics

All four metrics are computed directly from MIDI files. Use `pretty_midi` to extract note information.

Pitch Histogram Similarity

$$H(p, q) = \sum_{i=1}^{12} |p_i - q_i|$$

For each MIDI file, map every note to its pitch class via `pitch % 12`, accumulate counts in a 12-element array, and normalize by total note count to obtain a distribution. Compute the L_1 distance between the distribution of a generated file q and a reference real file p. A score of 0 indicates identical pitch class usage; the maximum possible value is 2. Lower scores indicate greater similarity to real music.

Rhythm Diversity Score

$$D_{\text{rhythm}} = \frac{\#\text{ unique durations}}{\#\text{ total notes}}$$

For each note, compute duration as `end - start` in seconds, then quantize to the nearest 50 milliseconds to avoid floating-point noise inflating uniqueness counts. Divide the number of distinct quantized durations by the total note count. Higher values indicate greater rhythmic variety.

Repetition Ratio

$$R = \frac{\#\text{ repeated patterns}}{\#\text{ total patterns}}$$

Extract the sorted-by-onset pitch sequence from the generated MIDI. Compute all overlapping n-grams of length 4. Count how many n-grams appear more than once. The repetition ratio is that count divided by total n-grams. Values between 0.1 and 0.5 are typical for coherent classical music; values near 0 or 1 suggest incoherence or a degenerate model.

Perplexity (Task 3)

Compute average cross-entropy per token on the validation or test set, then exponentiate. Lower perplexity means the model assigns higher probability to held-out sequences. The project specification example table lists approximately 12.5 for a well-trained Transformer and 11.2 after RLHF fine-tuning.

Human Listening Score (All Tasks)

Recruit at least 10 participants. Present each MIDI sample converted to audio and ask for a rating from 1 (poor) to 5 (excellent). Average ratings across participants and samples for each model. This score is required for the comparison table across all four tasks and both baselines.

Results Table

Produce a comparison table with one row per model (Random Generator, Markov Chain, Task 1, Task 2, Task 3, Task 4) and columns for: reconstruction loss, perplexity, rhythm diversity, human score, and genre control. This matches the format of Table 3 in the project specification.

Step 10: Baseline Models

The project requires comparison against at least two baselines.

Random Note Generator. Generate a MIDI file by sampling pitches uniformly from the 88-key piano range and durations from a fixed set of values, placing notes at random onset times within a fixed window. This is the lower bound for all metrics.

Markov Chain Model. Build a first-order transition matrix from training data by counting, for each pitch i, how often it is followed by pitch j. Normalize each row to obtain transition probabilities. Generate a sequence by sampling iteratively from the row corresponding to the current pitch. Use the empirical duration distribution from training data to assign note lengths. This baseline captures local melodic dependencies but no long-range structure.

MuseGAN is listed as an optional third baseline in the project specification. It is more involved to implement; attempt it only if time permits.

Compute all metrics for both required baselines using the same functions applied to model outputs.

Step 11: Loss Curves and Plots

Record training and validation loss at the end of every epoch for Tasks 1, 2, and 3. Plot both on shared axes with a solid line for training and a dashed line for validation.

For Task 2, produce a second plot showing KL loss and reconstruction loss separately over epochs. With correct KL annealing, the KL loss remains near zero during the warmup phase and rises gradually as β increases, while reconstruction loss falls throughout.

For Task 3, plot perplexity rather than raw cross-entropy for readability.

Save all plots as PDF or high-resolution PNG for inclusion in the final report.

Step 12: MIDI Export and Verification

Before submitting generated MIDI files, verify each one by loading it and checking: it loads without error, it contains at least 50 notes, and it has a duration of at least 5 seconds. Discard and regenerate samples that fail any check.

For listening tests, convert MIDI to audio using one of the following free tools.

- **MuseScore** (`https://musescore.org`): import the MIDI and export to MP3. Also provides a piano-roll view for visual inspection.
- **FluidSynth** (command-line): renders MIDI to WAV using a soundfont. Available via standard package managers.
- **GarageBand** (macOS only): drag and drop the MIDI file for immediate playback.

Troubleshooting: Common Failure Cases

This section catalogues the failure modes that arise across all four tasks. Each entry states the observable symptom, the underlying cause, and the corrective action. Failures are grouped by

the stage at which they appear.

A. Piano-Roll Representation Failures (Tasks 1 and 2)

Reconstruction Output is Near-Zero Everywhere

Symptom. Model output values are uniformly close to zero regardless of input. Generated MIDI files contain very few notes or are entirely silent.

Cause. Piano-roll matrices are approximately 97–98% zeros. Under MSE loss, predicting zero everywhere achieves a low loss value without learning any note activations. The gradient contribution from the small number of active cells is dominated by the gradient from the silent majority.

Fix. Replace MSE with a class-imbalance-aware loss such as Focal Loss with an appropriate γ and positive class weight, or `BCEWithLogitsLoss` with `pos_weight` set to the negative-to-positive cell ratio. Additionally lower the binarization threshold during generation below 0.5, since the model will still tend to underestimate note probabilities even after the loss change.

BCE Loss Plateaus at a Constant Value

Symptom. Both training and validation loss plateau immediately at a fixed value and show no further decrease over many epochs. The model produces the same output probability for every cell regardless of input.

Cause. Under binary cross-entropy with no class weighting, the optimal constant prediction on highly imbalanced data is a value close to the prior probability of the positive class. The model settles at this constant-output equilibrium because the gradient from the rare active cells is insufficient to move parameters away from it.

Fix. Add a positive class weight to `BCEWithLogitsLoss` proportional to the ratio of negative to positive cells. If loss does not decrease within several epochs after adding the weight, replace the loss entirely with Focal Loss.

Unstable Training After Adding Positive Class Weight

Symptom. After adding `pos_weight`, training loss oscillates sharply or diverges. Gradient norms are abnormally large.

Cause. An excessively large weight (e.g., 100 or above) amplifies the gradient from positive cells to the point of instability.

Fix. Restrict `pos_weight` to the range 10–30. Apply gradient clipping with a maximum norm of 1.0. If divergence persists, reduce the learning rate by a factor of 10.

Sigmoid Applied at Output Layer

Symptom. Training is numerically unstable from the start, or loss values are unexpectedly large or `NaN`.

Cause. Applying sigmoid in the final layer and then passing the result to `BCEWithLogitsLoss` causes a double sigmoid, which distorts gradients. The loss function internally applies its own sigmoid and expects raw logits as input.

Fix. Remove any activation function from the decoder output layer. Pass raw logits to the loss function. Apply sigmoid explicitly only during inference when converting outputs to probabilities.

B. LSTM Autoencoder Failures (Task 1)

Encoder Hidden State Carries No Useful Information

Symptom. Reconstruction loss decreases slightly but generated samples bear no relationship to the training data. Sampling different latent codes produces outputs that are nearly identical.

Cause. The encoder LSTM is not being used correctly. A common mistake is taking the output sequence rather than the final hidden state as the latent code, which does not produce a compact fixed-length representation.

Fix. Use the hidden state of the last LSTM layer at the final time step as the latent vector z. In PyTorch this corresponds to indexing `h_n[-1]` from the LSTM output tuple, not the full output tensor.

Decoder Cannot Condition on Latent Code

Symptom. The decoder produces the same output regardless of which latent code is provided. Reconstruction loss is low but generation from random z samples always yields the same pattern.

Cause. The latent code is not being injected into the decoder correctly. If z is only used to initialize the hidden state of the decoder LSTM but the input at each time step is zeros or a learned constant, the decoder may ignore z after the first few steps.

Fix. Repeat z across all time steps and concatenate it with the decoder input at each step, so that the latent code is visible throughout the entire decoding process.

Overfitting with Low Training Loss and High Validation Loss

Symptom. Training loss decreases steadily but validation loss increases after an initial decrease. Generated samples from training pieces are reasonable but samples from held-out pieces are incoherent.

Cause. The model has memorized training examples rather than learning a generalizable latent representation. This is more likely with large hidden dimensions and small datasets.

Fix. Add dropout (rate 0.2–0.3) after each LSTM layer. Reduce hidden dimension. Increase the amount of training data by using more files from the training split.

C. VAE-Specific Failures (Task 2)

Posterior Collapse

Symptom. KL divergence drops to near zero within the first few epochs and stays there. The encoder outputs approximately $\mu = 0$ and $\log \sigma^2 = 0$ for all inputs, meaning the posterior equals the prior regardless of the input.

Cause. The decoder is sufficiently powerful to reconstruct inputs without using z. As a result, the encoder receives no gradient pressure to encode information, and $q_\phi(z|X)$ collapses to $\mathcal{N}(0, I)$.

Fix. Apply KL annealing: set $\beta = 0$ for an initial warmup period and increase linearly to 1 thereafter. If collapse persists, reduce decoder capacity by removing one LSTM layer or halving the hidden dimension.

Reconstruction Loss Does Not Decrease in the VAE

Symptom. Reconstruction loss is flat from the beginning of training even though the plain autoencoder from Task 1 converged normally.

Cause. The KL term is active from epoch 1 and penalizes any deviation of the posterior from the prior before the encoder has learned a useful representation.

Fix. Apply KL annealing with $\beta = 0$ initially. Do not increase β until reconstruction loss has been consistently decreasing for several epochs.

Latent Space Discontinuity

Symptom. Decoding from random samples $z \sim \mathcal{N}(0, I)$ produces incoherent outputs even though reconstructions of real inputs are reasonable. Latent interpolation between two pieces produces an abrupt transition rather than a smooth one.

Cause. The KL weight β is too small or the annealing warmup is too short, so the encoder encodes inputs to widely separated regions of latent space that are far from the prior.

Fix. Increase the final value of β or extend the annealing warmup. Verify that the KL loss is non-trivially positive at the end of training (not near zero and not extremely large).

Generated Outputs are All Identical

Symptom. Multiple samples drawn from $\mathcal{N}(0, I)$ decode to the same or very similar outputs.

Cause. The decoder has learned to ignore z and always produces its most common training pattern. The latent space has no meaningful structure.

Fix. Increase β to force stronger regularization. Inspect encoder outputs by plotting the distribution of μ values across 50 training samples; they should be spread across the latent space rather than clustered at the origin.

D. General Training Failures (Tasks 1 and 2)

Very Slow Learning

Symptom. Loss decreases very slowly. After 50 or more epochs the model still produces mostly silent or incoherent output.

Cause. Common causes are: learning rate too low, un-filtered training windows that are mostly silence, or batch size too small to provide stable gradient estimates.

Fix. Use Adam with a learning rate of 10^{-3} as a starting point. Confirm that sparse window filtering is active during preprocessing. Reduce window length if using sequences of 300 steps or more. Verify that the DataLoader shuffles data each epoch.

Repetitive Output

Symptom. Generated MIDI files contain the same short pitch pattern repeated for the entire duration. The repetition ratio metric is above 0.8.

Cause. The decoder has converged to a degenerate mapping that produces the same pattern for most or all latent codes. This can follow from a poorly regularized latent space or an insufficient loss function.

Fix. Verify that the latent space has structure (see latent space inspection above). Increase latent dimension. For the VAE, confirm that KL annealing has produced a non-zero KL loss at convergence.

Loss is NaN After a Few Iterations

Symptom. Loss becomes `NaN` or `inf` after 1–10 training steps.

Cause. Exploding gradients, division by zero in a normalization step, or logarithm of zero in the loss computation.

Fix. Add gradient clipping with maximum norm 1.0 before the optimizer step. Check that `BCEWithLogitsLoss` is receiving raw logits, not probabilities clipped to 0 or 1. For the VAE, ensure $\sigma^2 > 0$ by computing $\sigma = \exp(0.5 \cdot \log \sigma^2)$ rather than taking the square root directly.

E. Transformer Failures (Task 3)

Perplexity Does Not Decrease

Symptom. Validation perplexity remains high throughout training and does not decrease across epochs despite training loss falling normally.

Cause. There are two distinct scenarios. First, if the causal mask is missing entirely, the model can attend to future tokens during training, making the next-token prediction task trivially easy and producing very low training loss. At generation time, however, no future context is available, so the model produces incoherent output and perplexity on held-out data stays high. Second, if the positional embedding is absent, the model has no sense of token order and cannot learn sequential dependencies; both training and validation perplexity remain elevated.

Fix. Verify that a causal (upper-triangular) attention mask is applied at every self-attention layer. Confirm that positional embeddings are added to token embeddings before the first Transformer layer. If training loss is very low but validation perplexity is high, suspect the missing causal mask specifically.

Incoherent Long-Range Structure

Symptom. Short generated sequences (10–20 tokens) are locally plausible, but sequences of hundreds of tokens lose musical coherence and drift into random-sounding patterns.

Cause. The Transformer's attention mechanism can in principle attend to all previous positions, but in practice it may not learn long-range dependencies if the training sequences are short or the model is under-trained.

Fix. Train on longer token sequences (512–1024 tokens). Increase the number of attention heads and layers if model capacity is the bottleneck. Consider using relative positional encodings, which are better suited to music than absolute positional encodings.

Repetition in Autoregressive Generation

Symptom. During generation, the model enters a loop and repeats the same short token subsequence indefinitely.

Cause. Greedy decoding (always selecting the highest-probability next token) causes the model to reinforce its own recent outputs.

Fix. Use temperature sampling with temperature $T \in [0.8, 1.2]$ or top-k sampling with $k \in [5, 50]$ instead of greedy decoding. Higher temperature increases diversity; lower temperature increases coherence.

Out-of-Vocabulary Tokens During Generation

Symptom. The token-to-MIDI conversion step raises an error about unrecognized token IDs.

Cause. The model generates token IDs outside the vocabulary range, which can happen if the output projection layer is not masked to the vocabulary size or if special tokens (padding, end-of-sequence) are sampled during generation.

Fix. Mask special tokens (padding and end-of-sequence) from the sampling distribution during generation. Ensure the output linear layer has exactly `vocab_size` output units.

F. RLHF Failures (Task 4)

Reward Model Overfits to Surveyed Samples

Symptom. The reward model predicts high scores for all generated outputs, including clearly incoherent ones, after a few training iterations.

Cause. The reward model has been trained on too few human-labeled samples and has memorized the training ratings rather than learned a generalizable preference signal.

Fix. Collect ratings from more participants or for more generated samples before training the reward model. Regularize the reward model with dropout or weight decay.

Policy Gradient Reward Hacking

Symptom. The reward score predicted by the reward model increases rapidly, but actual human listeners rate the generated outputs as worse than before fine-tuning.

Cause. The generator has found a mode of output that scores highly on the reward model but does not correspond to genuine musical quality. This is known as reward hacking and is a fundamental challenge in RLHF.

Fix. Add a KL penalty between the fine-tuned policy p_θ and the original pretrained policy p_{θ_0} to prevent the generator from deviating too far: $J'(\theta) = \mathbb{E}[r] - \lambda\, D_{\mathrm{KL}}(p_\theta \| p_{\theta_0})$. Periodically re-evaluate generated outputs with human listeners rather than relying solely on the reward model.

High Variance in Policy Gradient Estimates

Symptom. Training is unstable. The loss fluctuates widely from iteration to iteration even with a small learning rate.

Cause. REINFORCE gradient estimates have high variance when the reward signal is not normalized.

Fix. Normalize rewards within each batch by subtracting the batch mean and dividing by the batch standard deviation before computing the gradient. Use a small learning rate (10^{-5} to 10^{-4}) for fine-tuning.

G. Data Pipeline Failures (All Tasks)

MIDI Files Fail to Load

Symptom. Some MIDI files raise parsing errors when loaded with `pretty_midi`.

Cause. A small number of MIDI files in any large dataset may be malformed, have non-standard encodings, or contain events that the parser does not handle.

Fix. Wrap each file load in a try-except block and skip files that raise exceptions. Log skipped files for inspection.

Train/Validation Performance Gap is Absent

Symptom. Training and validation loss are identical throughout training.

Cause. The validation set is being used incorrectly, for example by including validation files in the training DataLoader, or by evaluating on the training set by mistake.

Fix. Filter the metadata CSV by the `split` column strictly. Confirm that the training DataLoader contains only rows where `split == 'train'` and the validation DataLoader contains only rows where `split == 'validation'`.

Generated MIDI is Valid but Unplayable

Symptom. The MIDI file is written successfully and loads without error, but playback produces no sound or only a single continuous tone.

Cause. Note start and end times are identical (zero-duration notes), notes have velocity of zero, or all notes are assigned to a non-piano MIDI program number that has no sound in the playback soundfont.

Fix. Set a fixed positive velocity (e.g., 80) when constructing `pretty_midi.Note` objects. Ensure note duration is at least 1/`fs` seconds. Set the instrument program number to 0 (Acoustic Grand Piano).

Priority fix sequence for Tasks 1 and 2.

1. Filter sparse windows during preprocessing; discard windows that are predominantly silence.
2. Remove any activation from the decoder output layer; pass raw logits to the loss function.
3. Replace MSE or plain BCE with Focal Loss with an appropriate positive class weight.
4. Lower the generation binarization threshold below 0.5 to recover suppressed note activations.
5. For the VAE: apply KL annealing, starting from $\beta = 0$ and ramping linearly to 1 over a warmup period.
6. If gradients are unstable: add gradient clipping and reduce the learning rate.
7. If piano-roll sparsity problems persist: switch to token-based input with `miditok` REMI.

Priority fix sequence for Task 3.

1. Verify that a causal attention mask is applied at every self-attention layer; without it, the model can attend to future tokens during training, training loss falls quickly, but validation perplexity remains high and generation is incoherent.
2. Confirm that positional embeddings are added to token embeddings before the first Transformer layer.
3. If special tokens (padding, end-of-sequence) are being sampled during generation, mask them from the output distribution.
4. Replace greedy decoding with temperature or top-k sampling to prevent autoregressive repetition loops.
5. If long-range coherence is poor, train on longer token sequences and consider relative

positional encodings.

6. If perplexity on validation is much higher than on training, add dropout to attention and feed-forward sublayers.

Reference Resources

1. MAESTRO Dataset, download page (Google Magenta):
   `https://magenta.tensorflow.org/datasets/maestro`
2. `pretty_midi`: MIDI parsing and piano-roll extraction:
   `https://github.com/craffel/pretty-midi`
3. `miditok`: MIDI tokenization library:
   `https://github.com/Natooz/MidiTok`
4. Keras tutorial: Music Generation with Transformer on MAESTRO (2024):
   `https://keras.io/examples/generative/midi_generation_with_transformer/`
5. Piano-VAE: PyTorch LSTM-VAE with piano-roll pipeline:
   `https://github.com/Khamies/Piano-VAE`
6. LSTM, VAE, and GAN music generation with MAESTRO preprocessing:
   `https://github.com/alex21th/music-generation`
7. MusicVAE: Hierarchical VAE for Music in PyTorch (DTU):
   `https://github.com/Variational-Autoencoder/MusicVAE`
8. MusDr: Quantitative evaluation metrics for symbolic music (ISMIR 2020):
   `https://github.com/slSeanWU/MusDr`
9. music21: Symbolic music analysis toolkit (MIT):
   `https://web.mit.edu/music21/`
10. MuseScore: MIDI playback, audio export, and notation viewer:
    `https://musescore.org/en/download`